To address your clarification: This setup uses the VPN solely for your devices (Mac, iPhone, Android tablet) to access the internal VMs in your GCP VPC network. The VMs themselves won't have their outbound internet traffic routed through the VPN— their traffic remains unchanged and uses GCP's standard outbound paths. We'll configure a split-tunnel on the clients, so only traffic destined for the VPC (and VPN subnet) goes through the tunnel; your devices' general internet access stays direct and unaffected.

The WireGuard server VM will act as a gateway, forwarding traffic from connected devices to the internal VMs. We'll use NAT on the server for simplicity (so you don't need to add static routes or WireGuard on every internal VM), but this only affects inbound access from your devices—VMs' outbound remains normal. All internal VMs should be created without external IPs for security.

Step 1: Identify Your VPC Subnet

In Google Cloud Console > VPC networks > Select your VPC (likely "default") > Subnets tab. Note the IP range (e.g., 10.128.0.0/20). Use this in configs below.

Step 2: Create VMs

- **VPN Server VM**: As before, with external IP, allow UDP 51820 in firewall.
- **Internal VMs**: Create similarly, but under Networking > No external IP. Ensure same VPC and region/zone for simplicity. Allow internal traffic (default GCP behavior).

Step 3: Install and Configure WireGuard on the Server VM

SSH in and run:

1. `sudo apt update && sudo apt upgrade -y`
2. `sudo apt install wireguard -y`
3. Generate keys: `wg genkey | tee private.key | wg pubkey > public.key`
4. Find interface name: `ip link show` (usually eth0 or ens4).
5. Create `/etc/wireguard/wg0.conf`:

```text
[Interface]
Address = 10.0.0.1/24
PrivateKey = <server private key>
ListenPort = 51820
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A POSTRO
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat -D POST
```

   - This enables forwarding and NAT, so traffic from devices appears to come from the server's internal IP when reaching other VMs.

6. Enable IP forwarding: Add `net.ipv4.ip_forward=1` to `/etc/sysctl.conf`, then `sudo sysctl -p`.
7. Start: `sudo wg-quick up wg0` and `sudo systemctl enable wg-quick@wg0`.

Step 4: Client Setup (One Per Device)

For each:

1. On server, generate client keys: `wg genkey | tee client-private.key | wg pubkey > client-public.key`
2. Add to server's `wg0.conf`:

```text
[Peer]
PublicKey = <client public key>
AllowedIPs = 10.0.0.2/32  # Increment per client: .3, .4, etc.
```

3. Restart server: `sudo wg-quick down wg0 && sudo wg-quick up wg0`
4. Client config (e.g., client.conf):

```text
[Interface]
Address = 10.0.0.2/24
PrivateKey = <client private key>

[Peer]
PublicKey = <server public key>
AllowedIPs = 10.0.0.0/24, 10.128.0.0/20  # VPN subnet + your VPC subnet
Endpoint = <server external IP>:51820
PersistentKeepalive = 25
```

   - This is the split-tunnel: only routes VPC/VPN traffic through the tunnel.

5. Import to apps as before.

Step 5: Testing Access

- Connect a device to VPN.
- From device, ping/ssh to an internal VM's private IP (find via `ip addr` on VM or GCP console).
- If issues: Check GCP firewall allows internal traffic (default yes). Verify NAT/forwarding with `iptables -t nat -L` and `wg show`.
- VMs' outbound internet (e.g., `curl ifconfig.me` from a VM) remains direct via GCP, not through WireGuard.

This keeps things minimal and open-source. No changes needed on internal VMs. Data costs apply only for traffic through the server.